

# Proyecto programacion/base de datos

Presentación..... 2

Objetivos..... 2

Alcance:..... 2

Roles:..... 3

Posibles riesgos:..... 3

Fortalezas:..... 4

Debilidades:..... 4

Oportunidades:..... 4

Amenazas:..... 4

Normativas del equipo:..... 4

Análisis de interesados:..... 6

Estrategias:..... 6

Requerimientos no funcionales:..... 7

Bibliografía..... 8

Casos de uso:..... 10

Capa Grafica(Figma)..... 10

Diagrama de clases:..... 11

DER (Diagrama Entidad Relación):..... 12

Pasaje a 3FN..... 13

Dependencias Funcionales (DF) explícitas..... 13

Consultas SQL..... 14

## Presentación

Hola. Somos la Tiktok Rizz Army, y nuestro equipo está conformado por Santiago Yañez, Franco Pesce y Fabricio Pintos.

## Objetivos

El proyecto trata de desarrollar una aplicación que permita a los docentes avisar de su inasistencia debido a licencias médicas o problemas personales.

La aplicación será programada en Java orientado a objetos, contará con una interfaz gráfica construida con Swing (NetBeans) y gestionará los datos mediante una base de datos relacional MySQL.

La información visible al público incluirá el nombre del docente, turno, grupos afectados y fecha de

finalización de la licencia.

### **Alcance:**

Todos los docentes, directivos, adscritos y secretarios que trabajan en secundaria pública o u.t.u del país (Uruguay)

### **Roles:**

- Santiago Yáñez: Programador back end
- Fabricio pintos: Diseñador
- Franco pesce: encargado de base de datos / escritor

Los roles fueron basados en las capacidades de los integrantes, las cuales son en el caso de Santiago la programación, el será el mayor responsable en cuanto al código. Franco, él se encargará de crear la base de datos y la documentación y por último en el de Fabricio siendo el diseño gráfico.

Intentamos resaltar las fortalezas de cada uno y llegamos a un acuerdo entre los tres participantes del equipo, por este motivo no hay un "líder" designado, el grupo tiene una estructura horizontal o de trabajo por consenso.

Esto quiere decir que todos tienen el mismo peso a la hora de decidir y nadie manda sobre los demás. Las tareas y las responsabilidades se reparten de forma pareja, y lo que se hace se define entre todos, buscando que haya acuerdo general.

Este tipo de organización favorece la participación y la cooperación, aunque a veces puede llevar más tiempo decidir porque todos tienen que estar de acuerdo para avanzar.

### **Posibles riesgos:**

- Tiempos de entrega

- Compromiso con el trabajo
- Insuficiencia de conocimientos de programación

**Fortalezas:**

- Nos comunicamos bien entre nosotros.
- Sabemos programar en Java y usar GitHub.
- Tenemos buena organización del trabajo.

**Debilidades:**

- Falta de experiencia técnica en ciertas áreas
- Algunos integrantes pueden no dominar tecnologías específicas
- Riesgo de sobrecarga del equipo( al combinar tareas de diseño, programación, pruebas y presentación, el grupo puede sufrir estrés o falta de coordinación.)
- Retrasos en la comunicación del equipo
- Problemas en la integración entre módulos o capas
- Riesgo de pérdida de datos

**Oportunidades:**

- Podemos aprender nuevas cosas.
- contamos con apoyo de los profesores

**Amenazas:**

- Poco tiempo para entregar el proyecto.
- Problemas con las computadoras del i.t.i

**Normativas del equipo:**

1. **Objetivo:** Este acuerdo busca dejar en claro el compromiso de todos los integrantes del grupo en el desarrollo de la aplicación. Cada miembro debe participar activamente, colaborar con el resto y cumplir con los plazos acordados entre el equipo y los docentes. La idea es que el trabajo sea equitativo.

### **Participación activa**

- Se considera que un integrante participa activamente cuando:

1. Cumple con las tareas que se le asignan dentro del tiempo establecido.
2. Asiste y colabora en las reuniones del grupo.
3. Mantiene una comunicación constante con los demás integrantes.

### **Desinterés en el proyecto**

Se considera que un estudiante muestra desinterés cuando:

- No cumple con las tareas.
  - No participa en las reuniones.
  - No colabora en el desarrollo del proyecto o no entrega los avances que le tocan.
2. Mejoras activas: el equipo tendrá reuniones ya sean en línea o personalmente para avanzar en el proyecto después de cada entrega y antes de la primera entrega estas reuniones serán semanales para avanzar con el proyecto y se realizarán todos los viernes de 6:00pm - 8:00pm y no serán registradas en otro lugar aparte del grupo de whatsapp utilizado para la comunicación entre los integrantes del equipo.
  3. Gestión del tiempo: habrá un tiempo límite para tener lista cada entrega el cual es hasta el 10/11 antes de esta entrega habrá una en el mes de julio en el cual deberemos de entregar la documentación y el servidor creado para redes pesce será responsable de la documentación y la base de datos y yañez de la programación, lo primero que deberá estar terminado es la documentación.
  4. Resolución de conflictos: Los desacuerdos se resolverán mediante el diálogo

respetuoso y votación, si es necesario, de haber problemas en el equipo se intentará llegar a un acuerdo y si no es posible se le hablara a alguno de los docentes responsables y si aun asi no se llega a un acuerdo se expulsara a la persona es discordia, los motivos de dicha expulsion pueden ser, desinterés, malos tratos a otros integrantes, intento de sabotaje o borrar/editar cosas sin el consentimiento del resto del equipo.

5. Las reuniones serán una vez por semana o dos en caso de ser necesario

### **Análisis de interesados:**

**INTERESADOS(ANEP):** son los más interesados ya que recae en ellos el peso de que nuestra empresa (Tiktok rizz army) se encargue correctamente del desarrollo de la aplicación,

**Docentes:** no tienen poder sobre nosotros, pero son los que usarían la aplicación para marcar su asistencia/inasistencia

**Administrativos** (secretarios, dirección, y adscriptos) Los primeros se encargarían de controlar los datos y darle mantenimiento a la página y los adscriptos se encargarán de difundir los datos de inasistencias a alumnos, padres y directores además de marcar las inasistencias de los profesores para que posteriormente se le descuenten las horas no cumplidas del sueldo

### **Estrategias:**

**Ofensivas:** utilizar y exprimir nuestros conocimientos de programación al máximo

**Defensivas:** Utilizar la facilidad de uso y diseño amigable para eliminar la resistencia a usar la aplicación por parte de los docentes poco familiarizados con nuevas tecnologías

**Adaptación:** Hacer pruebas con compañeros para mejorar la aplicación

**Supervivencia:** hacer reuniones de emergencia en caso de escasez de tiempo

## Estrategia de recolección de datos:

Lista de preguntas simuladas para docentes/usuarios:

¿Qué datos necesitan registrar?

¿Cada cuánto tiempo se actualiza la información?

¿Qué les gustaría que la aplicación muestre al público (nombre del docente, fecha de la inasistencia.)

## Requerimientos no funcionales:

- 1.Las cédulas de identidad se ingresarán sin signos de puntuación.
- 2.Cada requerimiento debe ser atómico, mencionando solo una acción por vez.
- 3.La aplicación debe estar operativa las 24 horas del día.
- 4.Las fechas se escribirán en el formato aa/mm/dd.
- 5.La aplicación deberá responder en un tiempo máximo de tres segundos por operación.
- 6.La interfaz debe ser clara y fácil de usar para cualquier usuario.

## Requerimientos funcionales:

- 1.Inicio de sesión:

La aplicación debe permitir que los usuarios administradores ingresen mediante una cédula y una contraseña.

- 2.Validación de datos:

El sistema debe verificar que las credenciales ingresadas sean correctas antes de permitir el acceso.

- 3.Registro de inasistencias:

La aplicación debe permitir ingresar una inasistencia , registrando los datos correspondientes: nombre del docente, fecha de inicio y fecha de finalización.

- 5.Eliminación de inasistencias:

El sistema permitirá eliminar un registro de inasistencia cuando sea necesario.

- 6.Consulta de inasistencias:

La aplicación debe mostrar una lista con todas las inasistencias registradas, ordenadas por fecha o docente.

- 7. Visualización pública:

Los datos visibles al público deben incluir: nombre del docente, fecha de finalización de la licencia.

- 8. Gestión de usuarios:

El sistema debe permitir agregar, modificar o eliminar usuarios con permisos de administrador.

- 9. Control de acceso:

Solo los usuarios administradores podrán agregar, modificar o eliminar inasistencias; los demás solo podrán consultar la información.

- 10. Cierre de sesión:

El sistema debe permitir cerrar la sesión en cualquier momento y volver al inicio de forma segura.

- 11. Diseño de red en packet tracer:

El diseño debe mostrar cómo el equipo del administrador se conecta a una base de datos en un servidor local (LAN).

Debe incluir router, switch, PC del usuario y servidor MySQL simulado.

Cada requerimiento debe ser atómico, mencionando solo una acción por vez.

## Bibliografía

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2.<sup>a</sup> ed.). Addison-Wesley.

Bolívar, A. (2012). Liderazgo y gestión del cambio en los centros educativos. Ediciones Morata.

Deitel, P. J., & Deitel, H. M. (2017). Java: How to Program (11.<sup>a</sup> ed.). Pearson Education.

Fowler, M. (2004). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3.<sup>a</sup> ed.). Addison-Wesley.

Fullan, M. (2014). The Principal: Three Keys to Maximizing Impact. Jossey-Bass.

González, M. E. (2016). Gestión de la calidad: Fundamentos y aplicaciones de la norma ISO 9001:2015. Editorial UOC.

International Organization for Standardization (ISO). (2015). ISO 9001:2015 – Quality management systems — Requirements. ISO.

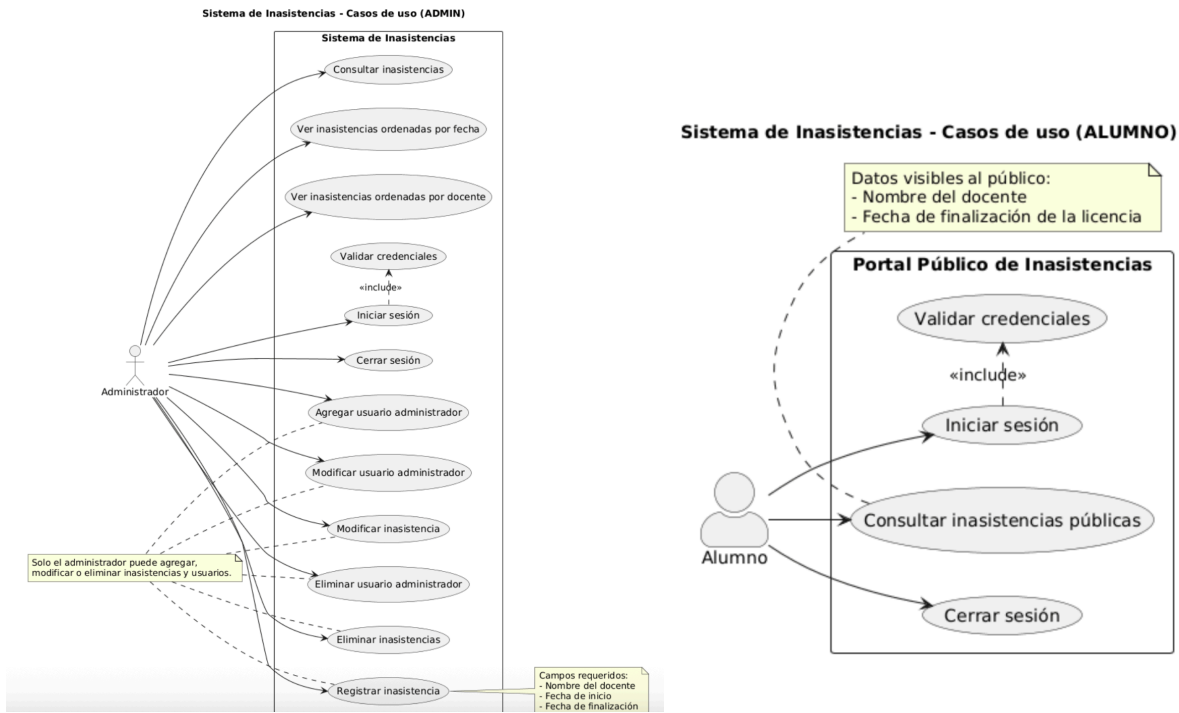
Oracle. (s.f.). Java Swing Tutorial. Recuperado de <https://docs.oracle.com/javase/tutorial/uiswing/>

Oracle. (s.f.). JDBC Overview. Recuperado de <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>

Schildt, H. (2018). Java: The Complete Reference (11.<sup>a</sup> ed.). McGraw-Hill Education.



## Casos de uso:



## Casos de uso del Administrador

### 1. Iniciar sesión

El actor principal es el administrador. El objetivo de este caso de uso es acceder al sistema mediante credenciales válidas.

Antes de iniciar, el administrador debe estar registrado. El flujo comienza cuando el usuario ingresa su cédula y contraseña. El sistema verifica que los datos sean correctos y, si lo son, permite el acceso a las funciones principales.

Como resultado, el administrador queda autenticado y habilitado para operar. En caso de error en las credenciales, el sistema muestra un mensaje indicando que el acceso fue denegado.

### 2. Validar credenciales

El actor principal es el sistema. Su propósito es confirmar que la cédula y la contraseña proporcionadas correspondan a un usuario válido.

La validación se realiza comparando los datos ingresados con los registros almacenados en la base de datos. Si existe coincidencia, el acceso se aprueba; en caso contrario, se rechaza.

La salida de este proceso determina si el usuario puede o no continuar al menú principal.

### 3. Registrar inasistencia

El administrador tiene la posibilidad de registrar una nueva inasistencia. Este proceso comienza luego de iniciar sesión.

El administrador selecciona la opción de registro e introduce la información requerida: nombre del docente, fecha de inicio y fecha de finalización.

El sistema guarda los datos en la base de datos. Si la operación es exitosa, la inasistencia queda registrada. En caso de que haya campos vacíos o un formato de fecha incorrecto, el sistema informa el error.

### 4. Modificar inasistencia

El objetivo de este caso de uso es permitir que el administrador actualice los datos de una inasistencia existente.

Una vez autenticado, el administrador elige el registro que desea modificar y realiza los cambios necesarios.

El sistema reemplaza los datos antiguos por los nuevos y confirma la actualización. El resultado final es que la información queda actualizada en la base de datos.

### 5. Eliminar inasistencia

El administrador puede eliminar una inasistencia registrada.

Para hacerlo, selecciona el registro correspondiente y confirma la acción. El sistema procede a borrar los datos de forma definitiva.

Como resultado, la inasistencia deja de estar disponible en el sistema. Si se intenta eliminar un registro inexistente, el sistema notifica el error.

### 6. Consultar inasistencias

El administrador puede consultar todas las inasistencias registradas.

El sistema muestra la lista completa y permite ordenarla por fecha o por nombre del docente.

Este caso de uso tiene como finalidad facilitar el control y seguimiento de las ausencias registradas.

### 7. Gestión de usuarios (agregar, modificar y eliminar)

El administrador tiene acceso a la gestión de cuentas con permisos administrativos.

Puede agregar nuevos usuarios, modificar los datos de los existentes o eliminarlos.

Cada una de estas acciones requiere autenticación previa y se registra en la base de datos.

De esta manera, se mantiene actualizado el control de accesos dentro del sistema.

### 8. Cerrar sesión

El administrador puede cerrar su sesión en cualquier momento.

Al hacerlo, el sistema borra los datos temporales de autenticación y regresa a la pantalla inicial.

La sesión queda finalizada de forma segura, y el usuario deberá volver a ingresar sus credenciales para acceder nuevamente.

### **casos de uso del alumno:**

### 1. Iniciar sesión

El alumno debe autenticarse antes de utilizar el portal.

Para hacerlo, ingresa su cédula y contraseña en la pantalla de inicio. El sistema valida los datos y, si son correctos, habilita el acceso a las funciones de consulta.

Si las credenciales no coinciden con las registradas, se informa el error y no se permite el ingreso.

### 2. Consultar inasistencias públicas

Una vez autenticado, el alumno puede acceder a la lista de inasistencias públicas.

El sistema muestra la información disponible, que incluye el nombre del docente y la fecha de finalización de su licencia.

Este caso de uso tiene como objetivo garantizar la transparencia y la comunicación entre la institución y los estudiantes.

### 3. Ver inasistencias ordenadas por fecha o por docente

El alumno puede elegir cómo visualizar la información.

Selecciona el tipo de ordenamiento deseado y el sistema reorganiza la lista de acuerdo con ese criterio.

Esto le permite encontrar más fácilmente la información que necesita.

## Capa Grafica(Figma):

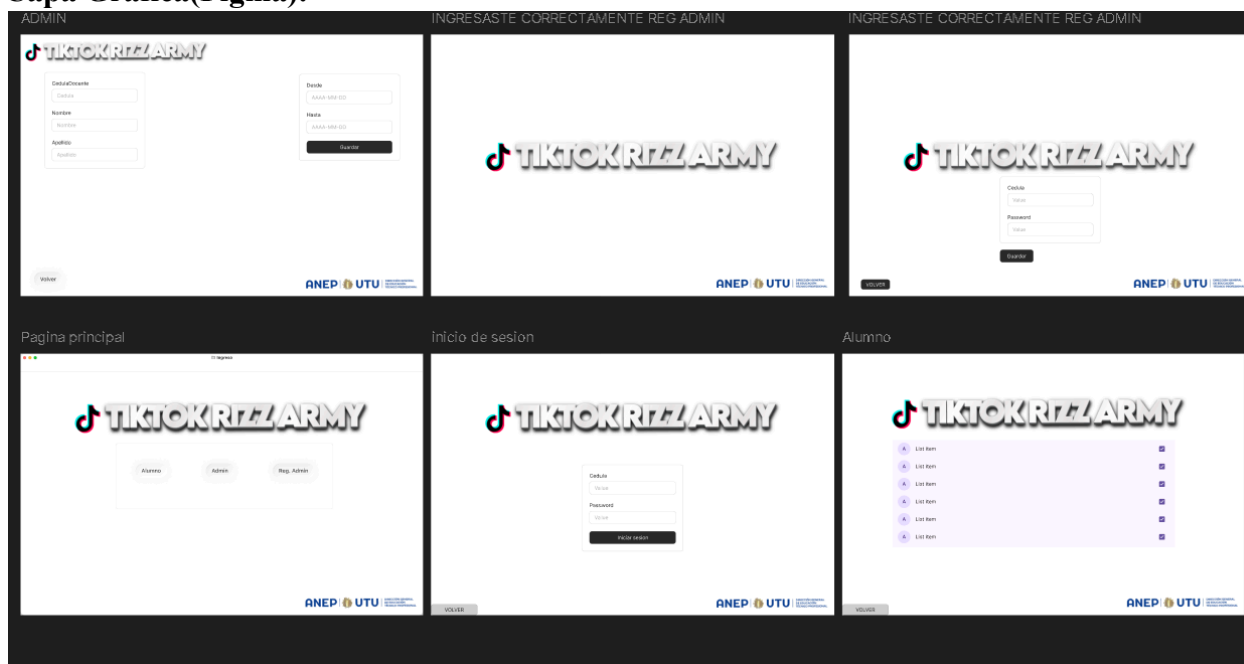
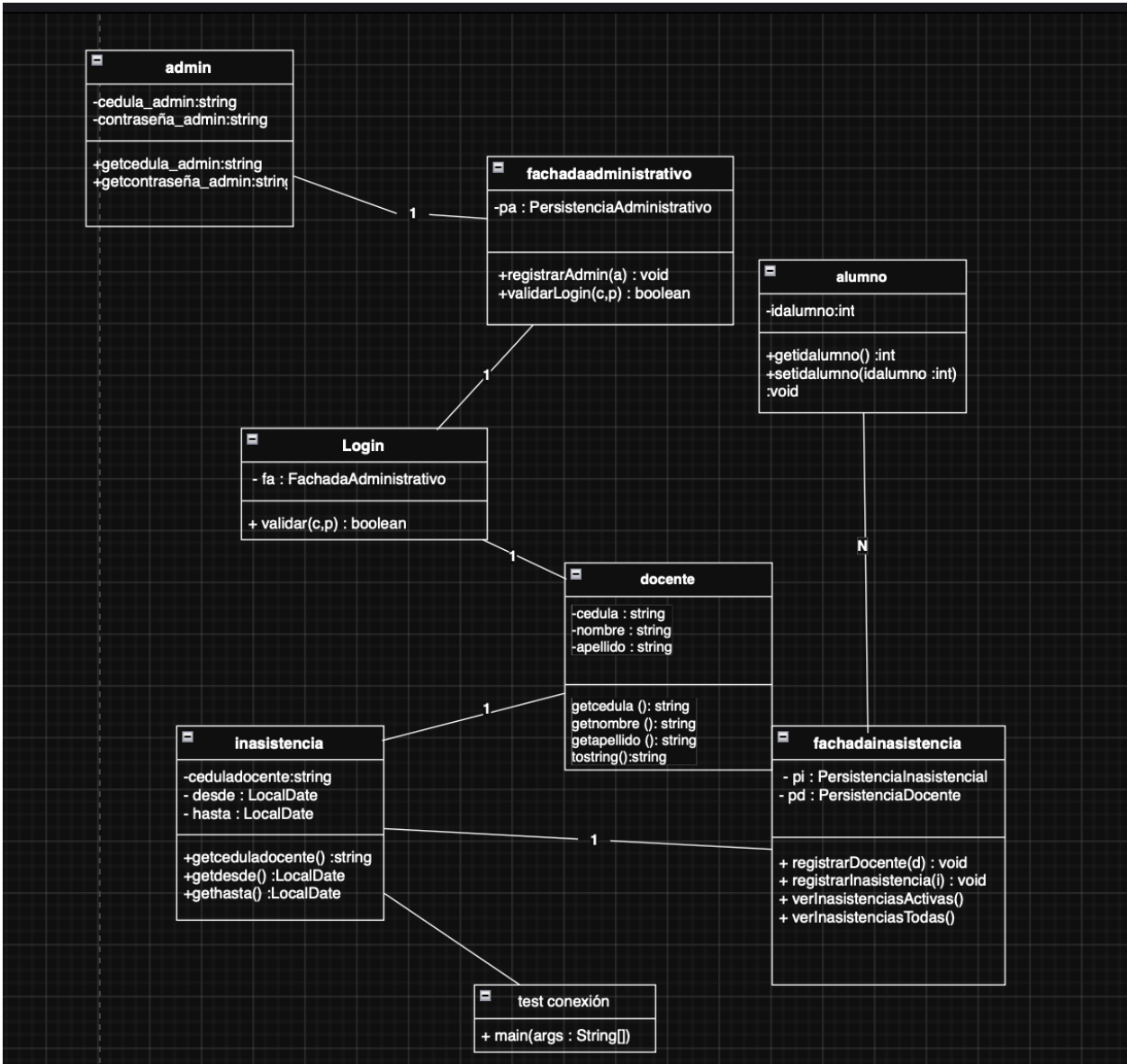
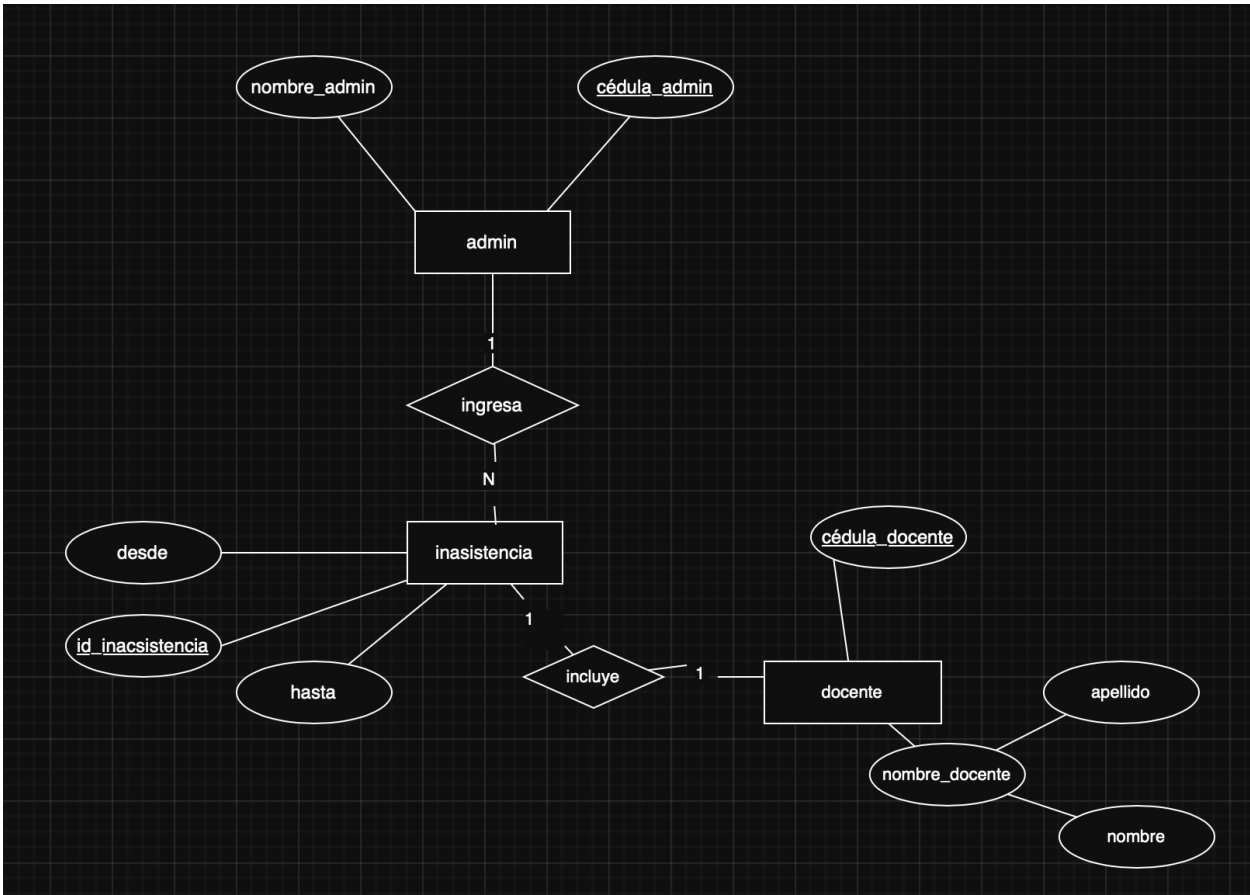


Diagrama de clases:



DER (Diagrama Entidad Relación):



## Pasaje a 3FN

Entidades y atributos:

- admin(cédula\_admin, nombre\_admin)
- docente(cédula\_docente, nombre, apellido)
- inasistencia(id\_inasistencia, desde, hasta )
- incluye(id\_inasistencia, cedula\_docente)
- ingresa(cedula\_admin, id\_inasistencia)

## Relaciones

- ingresa: admin (1 a N )ingresa inasistencia
- incluye: docente (1 a 1) incluye / tiene inasistencia

## Claves naturales razonables

- En docente: “cédula\_docente” identifica de forma única al docente.
- En admin: “cédula\_admin” identifica de forma única al admin.
- en inasistencia: “id\_inasistencia”

### Dependencias Funcionales (DF) explícitas

- En admin:  
cédula\_admin depende de nombre\_admin
- En docente:  
cédula\_docente depende de nombre, apellido

### Criterio 1FN:

todos los atributos deben ser atómicos, sin repeticiones ni listas.

resultado en 1fn:

ADMIN(cédula\_admin, nombre\_admin)

DOCENTE(cédula\_docente, nombre, apellido)

INASISTENCIA(id\_inasistencia, cédula\_admin, desde, hasta)

## Claves primarias:

- ADMIN cédula\_admin
- DOCENTE cédula\_docente
- INASISTENCIA id\_inasistencia

**Criterio 2FN:**

en tablas cuya PK sea compuesta, ningún atributo no clave debe depender de parte de esa PK.

En inasistencia la clave compuesta es (cédula\_docente, desde).

Los atributos hasta y cédula\_admin dependen de toda la clave, entonces no hay dependencia parcial.

conclusion: lo cumple

**Criterio 3FN:**

ningún atributo no clave debe depender transitivamente de la clave, en inasistencia cédula\_docente (nombre, apellido) es una dependencia transitiva si se guardan esos datos en la misma tabla.

Como los datos del docente y del admin están en sus propias tablas, no hay dependencia transitiva. entonces cumple con la 3FN.

**Consultas SQL**

1. Ver todas las inasistencias de un docente

$$\pi(\text{cedula\_docente, desde, hasta}) ((\text{Docente} \bowtie \text{incluye}) \bowtie \text{Inasistencia})$$

2. Ver qué alumnos visualizan las inasistencias de un docente

$$\pi(\text{cédula\_admin, nombre\_admin, nombre\_docente}) ((\text{Admin} \bowtie \text{Inasistencia}) \bowtie \text{Docente})$$

3. Ver qué administrativos registraron información de qué docentes

$$\pi(\text{cedula\_admin, contraseña, nombre\_docente}) ((\text{administrativo} \bowtie \text{ingresa}) \bowtie \text{docente})$$

4. Inasistencias dentro de un rango de fechas

$$\pi(\text{id\_inasistencia, nombre\_docente, nombre\_admin, desde, hasta}) ((\text{Inasistencia} \bowtie \text{Docente}) \bowtie$$

Admin)

5. Todos los docentes con sus inasistencias y los alumnos que pueden verlas

$$\pi(\text{id\_inacistencia, id\_alumno}) (((\text{docente} \bowtie \text{inasistencias}) \bowtie \text{visualiza}) \bowtie \text{alumno})$$